

# Practice Assignment 11.1: Hierarchical Planning with CrewAI

## Use Case: Corporate Event Planning Automation

Your organisation conducts an annual corporate event that includes keynote talks, breakout sessions, workshops, networking mixers, and vendor booths. Manually planning this across teams is extremely time-consuming and error-prone.

**Note:** This assignment mirrors the “*Italy Vacation Planner*” system [Hierarchical Planning family vacation.ipynb](#), but is tailored for an event management scenario. You can use that notebook as a reference for the overall structure (goal → hierarchical plan → execution), while applying the same ideas to corporate events instead of travel. This assignment does not require a solution set.

## Instructions

### Problem Statement

Your goal is to build a **Hierarchical Planning AI System** using **CrewAI** that:

- Breaks down the high-level event goal into a structured multi-level plan.
- Researches vendors, venues, and scheduling options.
- Builds a complete event budget.
- Creates a full event-day schedule.
- Produces a final consolidated event plan.

You are not expected to integrate real APIs or build a UI. Focus on clear planning logic, readable text outputs, and a crew of agents that work together in a sensible sequence.

### Required System Components

You must design and implement the following **CrewAI agents**:

#### 1. Event Planner Agent (Hierarchical Planner)

- Breaks the big goal into pillars: *venue, logistics, speakers, vendors, schedule, budget*.
- Generates sub-tasks and success metrics for each pillar.
- Produces a hierarchical plan (Level 1 → Level 2 → Level 3).

#### 2. Vendor Research Agent

- Research for:
  - Catering vendors (2–3 options).

- Audio/visual vendors.
- Seating & event décor providers.
- *Optional:* Stage setup companies.

### 3. Budget Analyst Agent

- Builds detailed cost estimates.
- Breaks down by category (venue, F&B, décor, A/V, labour, marketing).
- Suggests cost-saving strategies.
- Outputs total budget estimate & “must-book-first” items.

### 4. Schedule Designer Agent

- Creates a full-day event schedule.
- Includes keynote sessions, buffer times, breaks, workshops, and networking blocks.
- Includes speaker preparation windows.
- Should be realistic and time-bound.

### 5. Final Event Coordinator Agent

- Consolidates all outputs into one structured event plan.
- Includes booking checklist, timeline, risks, and contingencies.
- Produces a final formatted deliverable.

## Required Dynamic Inputs

Your CrewAI tasks must use placeholders:

- `{event_goal}` → e.g., “Plan a 1-day corporate innovation summit”.
- `{date}` → e.g., “2025-11-15”.
- `{attendee_count}` → e.g., “300 attendees”.
- `{preferences}` → e.g., “Premium look, vegetarian food options, mid-range budget”.

These must flow through multiple tasks, so that later agents (for example, Budget Analyst or Schedule Designer) can also “see” the event goal, date, attendee count, and preferences.

## Assignment Tasks

You must implement the following in Python using CrewAI:

### Task A — Build all 5 agents

- Define the five agents listed under “Required System Components”.
- Each must have:
  - Role
  - Goal
  - Backstory

- LLM instance

### Task B — Create 5 Tasks

- Define one CrewAI Task for each agent.
- Use structured `description` & `expected_output`.
- Inject dynamic placeholders correctly.

### Task C — Build the Crew

- Define list of agents.
- Define list of tasks.

### Task D — Run the System

- Pass dynamic inputs using:

```
crew.kickoff({"event_goal": ..., "date": ..., "attendee_count": ..., "preferences": ...})
```

- Print final consolidated output.

## Expected Output

Your final output should include:

- Hierarchical Event Plan.
- Vendor Research Summary.
- Budget Table + Total Estimated Cost.
- Full-Day Schedule.
- Consolidated Final Event Plan.

## Example Dynamic Input for Testing

Here are sample inputs you can use:

```
sample_inputs = {  
    "event_goal": "Organize a 1-day corporate innovation summit focusing on AI and automation.",  
    "date": "2025-11-15",  
    "attendee_count": "300 attendees",
```

```
"preferences": "Premium decor, vegetarian+vegan meal options, mid-range budget, interactive sessions"
}
```

## Outcome for the Learner

By completing this assignment, you will deepen your understanding of:

- Hierarchical planning.
- Multi-agent CrewAI orchestration.
- Dynamic task prompting.
- Planning → Research → Budgeting → Scheduling → Synthesis workflows.
- Exactly how to structure planning systems that move towards production-grade use cases.

## How to Approach the Project

Start by restating the event goal in your own words using the dynamic inputs, then focus on getting the five agents and their tasks defined in a simple, readable way. Keep each agent tightly scoped to its role, and let the Event Planner Agent drive the structure that the others follow. Use plain, well-commented Python and do not worry about perfect prompts or real-world pricing – the priority is to see the hierarchical plan, vendor research, budget, schedule, and final plan all flow through one CrewAI pipeline.

## Evaluation Checklist

- All five agents are implemented with Role, Goal, Backstory, and LLM instance.
- Each agent has a corresponding Task with structured description and expected\_output.
- Dynamic placeholders (`{event_goal}`, `{date}`, `{attendee_count}`, `{preferences}`) are correctly injected and passed through multiple tasks.
- The crew runs end-to-end with `crew.kickoff(...)` and produces the expected sections in the output.
- The hierarchical event plan, vendor summary, budget, schedule, and final plan are all present and coherent.

## Practice Guidelines:

This week, you will use **Vocareum** to practice building a **tool-using conversational agent** with **LangChain**, focusing on *memory*, simple *orchestration*, and (optionally) one *tool*, such as a *calculator* or *search engine*. You will prototype a short conversation (3–4 turns) that demonstrates the agent recalling earlier user context. You can also utilise the in-built GenAI assistant within Vocareum for assistance with prompts, debugging, and API-backed tasks. No submission is required for this activity; the focus is on fluency with agent memory and framework workflow.

#### To Use Chat Support for This Assignment (inside Vocareum):

- Open your Week 11 practice notebook in **Vocareum**. Use the left sidebar to open the **Assistant** panel.
- Ask for clarifications, debugging help, or code snippets directly in the **Assistant** chat while you implement memory, tools, and orchestration.
- Monitor your **GenAI Allowance/Budget**, as shown in the **Assistant** panel, and plan prompts accordingly.
- If you exhaust your allowance for a task, please contact the Emeritus support team.
- **API key access:** Your course-specific API key and usage notes are provided via the guide and Vocareum instructions. Follow those steps to locate your key when needed.
- You can find detailed instructions here: [Vocareum GenAI API Students Guide.pdf](#)

**Security warning:** Keep your API key *private*. Do not paste it in screenshots, chats, notebooks you share, or public repos. Prefer environment variables or provided key-management cells; never hard-code keys in source files. If you suspect exposure, rotate the key immediately per the guide.

#### To access Vocareum:

- Go to the *Getting Started* module on your Course homepage (just below Programme Orientation).
- Click the Vocareum link (labelled *Vocareum: Professional Certificate Programme in Agentic AI and Applications*).
- Alternatively, you can also access it from the Course menu on the left side of your screen.

**Note:** Working through problems independently is key to building confidence and developing a lasting understanding. This is a self-study activity for your practice and does not count towards programme completion. No submission is required. However, we highly recommend completing the activity on Vocareum to enhance your overall learning experience.